

LC 1631) Path with Minimum Effort

Authored by: Arpit Raj, LNMIIT Jaipur

Date: July 23, 2026

MMT

SAMSUNG

TCS

| Problem Breakdown

cost = diff between 2 adjacent cell values
we need min effort path

1 (start)	2	2
3	8	2
5	3	5 (end)

lets say aadya took path: $1 \rightarrow 3 \rightarrow 8 \rightarrow 2 \rightarrow 5$
then $\rightarrow$ **effort of this path = max (diff)**

i.e.

$$|1 - 3| = 2$$

$$|3 - 8| = 5$$

$$|8 - 2| = 6 \quad \rightarrow \text{max} = 6$$

$$|2 - 5| = 3$$

so we only care about largest jump

| Algorithm Concept

BFS wont work as shortest path isn't necessarily path with min effort

Instead of:

```
nextdist = currdist + cost
```

Use:

```
nextdist = max(currdist, cost)
```

C++ Implementation

```
class Solution {
public:
    int minimumEffortPath(vector<vector<int>>& heights) {
        int n = heights.size();
        int m = heights[0].size();

        vector<pair<int, int>> dir = {{-1, 0}, {0, -1}, {1, 0}, {0, 1}};

        priority_queue<pair<int, pair<int, int>>, vector<pair<int, pair<int, int>>>, greater<pair<int, pair<int, int>>>> pq;
        vector<vector<int>> dist(n, vector<int>(m, INT_MAX));

        pq.push({0, {0, 0}});
        dist[0][0] = 0;

        while (!pq.empty()) {
            int currcost = pq.top().first;
            int r = pq.top().second.first;
            int c = pq.top().second.second;

            pq.pop();
            if (currcost > dist[r][c]) continue;
            if (r == n - 1 && c == m - 1) return currcost;

            for (auto it : dir) {
                int nr = r + it.first;
                int nc = c + it.second;

                if (nr >= 0 && nc >= 0 && nr < n && nc < m) {
                    int cost = abs(heights[nr][nc] - heights[r][c]);
                    int newcost = max(currcost, cost);

                    if (newcost < dist[nr][nc]) {
                        dist[nr][nc] = newcost;
                        pq.push({newcost, {nr, nc}});
                    }
                }
            }
        }
        return -1;
    }
};
```

